

Security Audit

SilverSwap (DEX)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Conclusion	21
Our Methodology	22
Disclaimers	24
About Hashlock	25

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The SilverSwap team partnered with Hashlock to conduct a security audit of their SilverSwap smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

SilverSwap.io is a decentralized exchange (DEX) that enables users to trade and swap cryptocurrencies in a secure, non-custodial manner. The platform is built to facilitate decentralized trading without the involvement of intermediaries, offering users full control over their digital assets. By operating on blockchain technology, SilverSwap ensures transparency and security in transactions, while promoting a peer-to-peer trading model. Users can participate in liquidity pools, where they provide liquidity in exchange for potential rewards, helping to maintain the platform's fluidity and enabling seamless swaps of various digital assets.

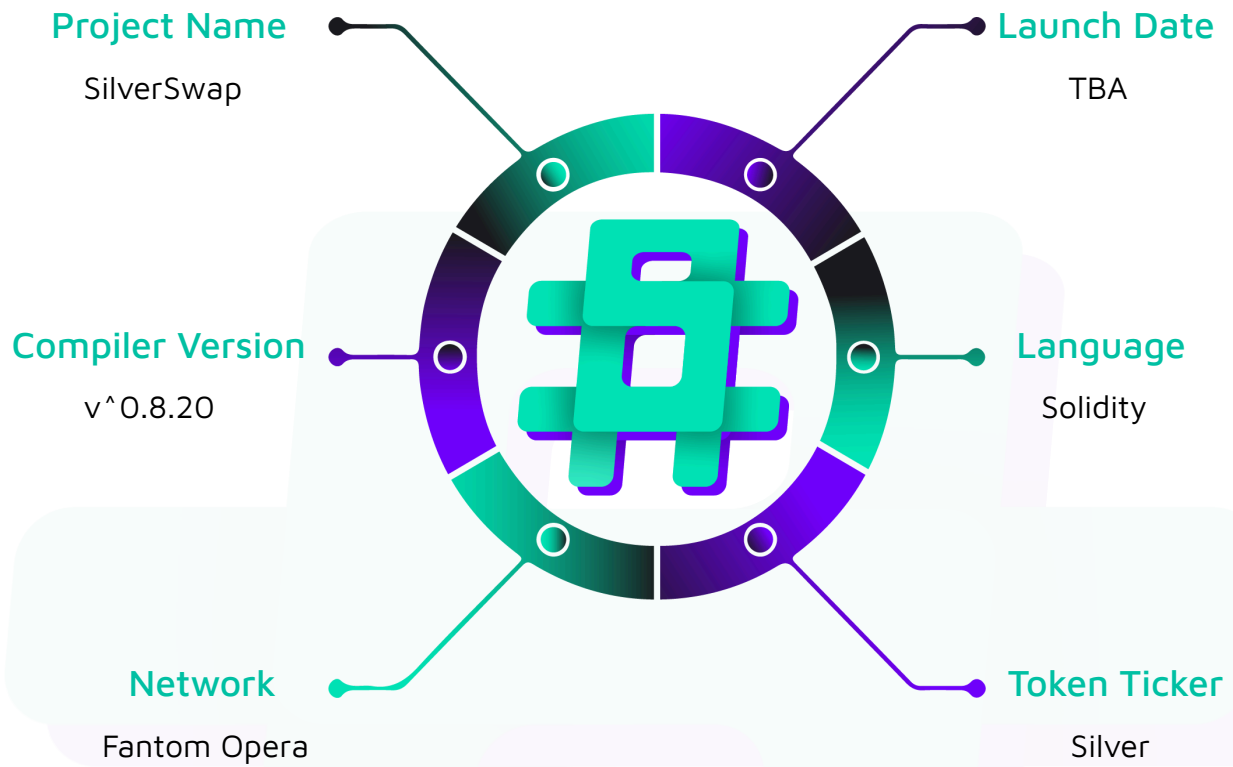
Project Name: SilverSwap

Compiler Version: ^0.8.20

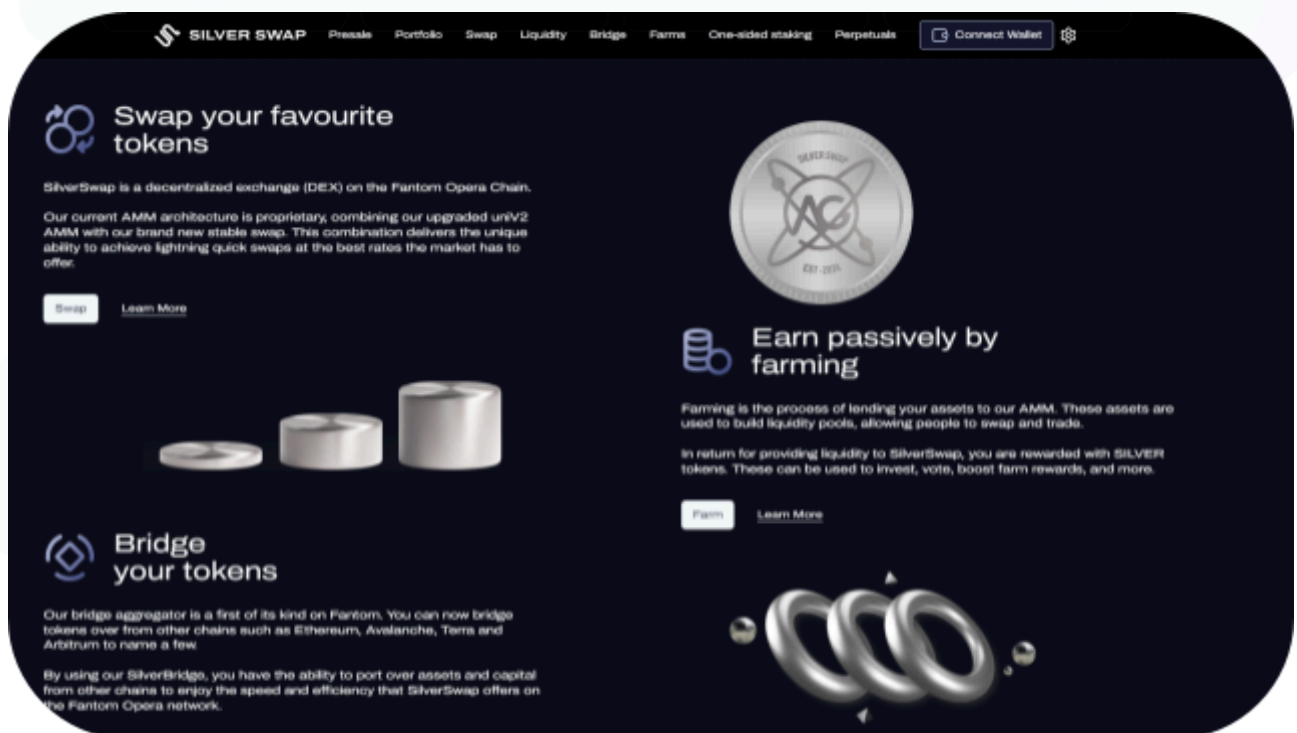
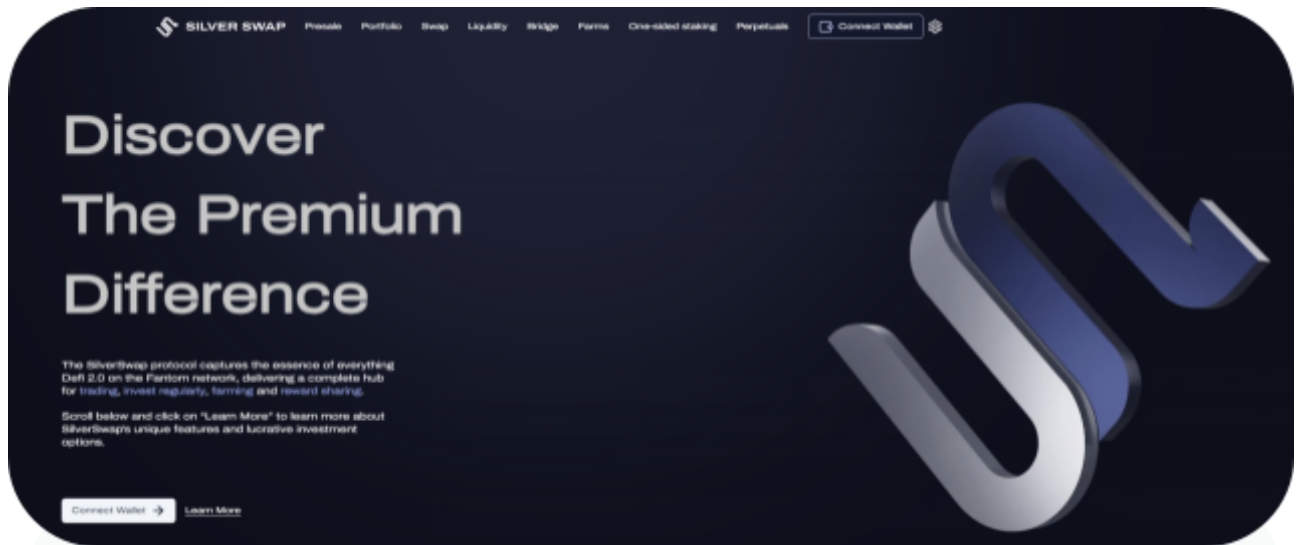
Website: <https://silverswap.io/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the SilverSwap project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	SilverSwap Smart Contracts
Platform	Ethereum / Solidity
Audit Date	December, 2024
Contract 1	SilverPresales.sol
Contract 1 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 2	Silver.sol
Contract 2 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 3	SilverMint.sol
Contract 3 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 4	SilverBurner.sol
Contract 4 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 5	SilverFees.sol
Contract 5 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 6	SilverFeesGiveaway.sol
Contract 6 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
Contract 7	SilverFlareProgram.sol
Contract 7 MD5 Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9
GitHub Commit Hash	5c1f995d87bffdefb3bcebc4990d7b085fdc68b9

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 Low severity vulnerabilities

1 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
- Tokenomics Breakdown Supply Max supply of tokens: 10,000,000. fully diluted after 9 years 300,000 reserved for pre-sale 200,000 reserved for initial liquidity 300,000 for team allocation/marketing 200,000 reserved for initial farm allocation 2739.72603 yield daily for remaining 9 years for farms	Contract achieves this functionality.
- Fees All swap fees earned to the protocol are swapped every 12 hours to \$AG or the network native token/wrapped variant (this is changeable on the fly per SilverSwap team's discretion) 12% of the earned fees are given to the SilverSwap team for funding, marketing, and other uses 3% allocated to giveaways/drawing feature or other social programs 85% allocated to various community interactive event such as URP (User-Rewards-Program), Flare, and Snatch The 85% remainder of funds is sent to two primary endpoints: Snatch payouts, and the Remainder Fund contract.	Contract achieves this functionality.
- snatch	Contract achieves this

Snatch is a 12-hour recurring event in which users use their \$AG to bid and win auctions for each and every white-listed pool on SilverSwap. When an auction is won, that user wins 50% of all of the protocol fees that were earned from that LP for the previous 12 hours. The \$AG used to win the bid is burned.	functionality.
- The remainder Fund acts as an escrow wallet that will, at launch, feed two primary directives: URP, and the Flare auction. The Remainder Fund kicks out to these two directives on a 12-hour recurring basis. The percentages of funds to URP and Flare are variable per the SilverSwap team's discretion.	Contract achieves this functionality.
- Flare Flare is a 12-hour recurring event in which users bid to create a buyback on a white-listed token of their choosing. The buyback is funded by the specified percentage of \$AG or wrapped native token in the remainder fund. The percentage of the remainder fund allocated to this is variable at our discretion. bought-back tokens are returned to the SilverSwap team. The \$AG used to win the bid is burned.	Contract achieves this functionality.
- Weekly giveaway The Weekly Giveaway function is a burn system in which users may burn 1 \$AG for a weekly ticket to earn an airdrop of that week's accumulation of the aforementioned 3% fee split to this function. No ticket max is described and all \$AG used to gain tickets are burned.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the SilverSwap project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the SilverSwap project's smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

- Algebra Finance
- Chainlink
- Gelato Networks

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] SilverPresales#_PRESALES_END - Presales end date is hardcoded

Description

In the `SilverPresales` contract, `PRESALES_END` is hardcoded as 1st October 2024 which is a past date. As the date is passed, it should be configured to a new date.

Recommendation

Pass the presale end date as a parameter in the constructor as part of the deployment.

Note

The Silverswap team informed Hashlock that the hardcoded date in the presale contract is intentional and does not impact functionality. The presale concluded on the specified date, which aligns with the hardcoded value. This design choice reflects the contract's deployment and completion timeline.

Status

Acknowledged

[L-02] SilverFeesGiveaway#_syncGiveaway - overrides the logic for setting value for giveawayEndTime in giveawayData data structure

Description

In the `syncGiveaway` function, when the giveaway has no participants or when it is over, the `applyEditedSettings` function is called to update the values for `giveawayData`. But, the logic in the `syncGiveaway` function overrides the setting applied by the `applyEditedSettings` function as it updates the `giveawayData` value directly.

```
function syncGiveaway() external onlySilverFees {
```

```

...

if (giveawayTimeCheck && giveawayData.numberOfParticipants > 0 && requestId == 0) {
...

} else if (giveawayTimeCheck && giveawayData.numberOfParticipants == 0) {

    applyEditedSettings();

    giveawayData.giveawayEndTime = silverFees.syncFeesLastSync() + giveawayTime;

} else if (giveawayTimeCheck && giveawayData.participationEnded && block.timestamp >

    ...

    applyEditedSettings();

    giveawayData.giveawayEndTime = silverFees.syncFeesLastSync() + giveawayTime;

}

}

```

Note, how after calling the `applyEditedSettings` function, the `giveawayEndTime` field of `giveawayData` is overwritten with new values bypassing the conditions implemented in the `applyEditedSettings` function.

Recommendation

As the rules for updating the `giveawayData` are implemented in `applyEditedSettings` function, it is recommended to update fields of `giveawayData` via `applyEditedSettings` function only.

Status

Resolved

[L-03] SilverFees#_executeFeeManagement - The return value of the transfer function on ERC20 is not being checked

Description

As a standard procedure, the return value of the `transfer()` function on the ERC20 token contract should be checked. The boolean returned by the `transfer()` function indicates the success and failure of the underlying transaction.

```
function executeFeesManagement() public onlyDedicatedMsgSender {  
  
    ...  
  
    if (feesManagementData.swapToWrappedToken) {  
  
        wrappedToken.transfer(teamMultisig, teamFees);  
  
        ...  
    } else {  
  
        silverToken.transfer(teamMultisig, teamFees);  
  
        ...  
    }  
  
}
```

Recommendation

As there are many ERC20 tokens and some of these contracts did not follow the standards and implementation. As such some of these contracts return false even when the underlying transaction was successful.

Due to these variations, openzeppelin has a SafeERC20 library.

Status

Resolved

[L-04] SilverFees#_executeFeeManagement - The return value of the transfer function on ERC20 is not being checked

Description

As a standard procedure, the return value of the `transfer()` function on the ERC20 token contract should be checked. The boolean returned by the `transfer()` function indicates the success and failure of the underlying transaction.

```
function executeFlare() public onlyDedicatedMsgSender {  
  
    ...  
  
    if (feesManagementData.swapToWrappedToken) {  
  
        ...  
  
        wrappedToken.transfer(flareProgramAddress, flareAmountProgram);  
  
    }  
  
    else {  
  
        ...  
  
        silverToken.transfer(flareProgramAddress, flareAmountProgram);  
  
    }  
  
    ...  
  
}
```

Recommendation

As there are many ERC20 tokens and some of these contracts did not follow the standards and implementation. As such some of these contracts return false even when the underlying transaction was successful.

Due to these variations, openzeppelin has a SafeERC20 library.

Status

Resolved

[L-05] SilverFees#_cancelTaskSnatch - cancelTaskSnatch does not clear the state

Description

`cancelTaskSnatch` function is used to cancel a task related to snatch flow. As a normal flow, when `startSystemSync` is called, the logic ensures to loop through all the `snatchPoolBids`, cancel the task, and also clear the state as below.

```
function startSyncSystem(uint256 time) public onlyOwner {
    ...
    for (uint256 i = 0; i < snatchPoolsBids.length; i++)
    {
        cancelTaskSnatch(snatchPoolsBids[i]);

        delete snatchData[snatchPoolsBids[i]].perPoolData;

        snatchData[snatchPoolsBids[i]].lastExecution = 0;

        snatchIsPoolBided[snatchPoolsBids[i]] = false;
    }

    delete snatchPoolsBids;

    ...
}
```

Note, how in the above for loop, after cancelling the bid for each element in the array, the logic clears the state level data related to the current element. This ensures that all the information related to the element is removed from the state.

On completion of the loop, deleting `snatchPoolsBids` clears the array completely.

As `cancelTaskSnatch` function is a public function, the problem arises when the owner ever calls it directly passing the pool address. In this case, as all the cleanup related to the pool address is not handled by the `cancelTaskSnatch` function, the state will be left until the next system sync call.

As a result, the `getAllBids` function may not return accurate data as canceled tasks will not be cleaned up, if `cancelTaskSnatch` was called directly by the owner.

```
function getAllBids(address user) public view returns (uint256) {  
  
    uint256 totalBids;  
  
    totalBids += flareLastBid(user);  
  
    for (uint256 i = 0; i < snatchPoolsBids.length; i++)  
        totalBids += snatchLastBid(user, snatchPoolsBids[i]);  
  
    return totalBids;  
  
}
```

Recommendation

The recommendation is to implement the cleanup of each `snatchPool Bid` in the `cancelTaskSnatch` function. This will ensure the state is cleaned up properly.

Note

The Silverswap team notified that concerns raised in this finding have already been discussed, and the conclusion was that no issues would arise from the identified matter. The decision to proceed was made with this understanding.

Status

Acknowledged

Gas

[G-01] SilverMint#_mintPresales - Task to mint for presales is canceled if `PRESALES_MINTED >= PRESALES_SUPPLY`

Description

The task to mint for presales is canceled if the `PRESALES_MINTED` is already greater than or equal to `PRESALES_SUPPLY`. While this check is done during the execution of the `mintPresales` function, it would be gas effective to check at the time of creating the task it.

Recommendation

Add the check-in `addPresaleUser` function to avoid creating the task and updating the storage variables. This will help save some gas.

Note

Silverswap is familiar with the finding and reported that the contract has already been deployed as there were no code changes following the last audit. Consequently, any adjustments or concerns raised post-deployment cannot be addressed retroactively within the current iteration of the contract.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Silverswap project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.